

MentorAid - Complete Technology Stack Documentation

Project Overview

MentorAid is an AI-powered Student Dropout Prediction System that helps educational institutions identify at-risk students and provide timely interventions.

1. Frontend Technologies

Core Framework

- **React 18.3.1** - Modern UI library for building interactive user interfaces
- **TypeScript 5.5.3** - Static typing for enhanced code quality and developer experience
- **Vite 5.4.2** - Fast build tool and development server

UI & Styling

- **Tailwind CSS 3.4.1** - Utility-first CSS framework for responsive design
- **PostCSS 8.4.35** - CSS processing tool
- **Autoprefixer 10.4.18** - Automatic vendor prefix management
- **Lucide React 0.344.0** - Modern icon library (500+ icons)
- **OGL 1.0.11** - Minimal WebGL library for 3D graphics and animations

Routing & Navigation

- **React Router DOM 7.9.1** - Client-side routing and navigation

Data Visualization

- **Recharts 3.2.1** - Composable charting library for React
- Line charts for student progress tracking
- Bar charts for risk level distribution
- Area charts for attendance trends

Authentication

- **@react-oauth/google 0.12.2** - Google OAuth 2.0 integration
- **Axios 1.13.2** - HTTP client with interceptors for JWT token management

State Management

- **React Context API** - Built-in state management
- **AuthContext** - User authentication state
- **ThemeContext** - Dark/Light mode switching

2. Backend Technologies

Core Framework

- **Flask 3.0.0** - Lightweight Python web framework
- **Gunicorn 21.2.0** - WSGI HTTP Server for production deployment

Cross-Origin Resource Sharing

- **Flask-CORS 4.0.0** - Handle cross-origin requests from frontend

Authentication & Security

- **Flask-JWT-Extended 4.6.0** - JWT token-based authentication
- Access tokens: 24-hour expiry
- Refresh tokens: 30-day expiry
- **Google Auth 2.27.0** - Google OAuth server-side verification
- **Google Auth OAuthLib 1.2.0** - OAuth 2.0 client library
- **Google Auth HTTPLib2 0.2.0** - HTTP transport adapter

Database Technologies

Primary Database (MongoDB)

- **PyMongo[srv] 4.6.1** - MongoDB driver with DNS seedlist support
- **Database:** MongoDB Atlas (Cloud)
- **Schema:** 4 Collections
- `users` - User authentication and profiles
- `students` - Student academic records
- `predictions` - ML prediction history
- `interventions` - Intervention tracking

Fallback Database (SQLite)

- **SQLite 3** - Embedded relational database
- **Database File:** `mentoraids.db`
- **Schema:** 4 Tables (mirrors MongoDB structure)
- Auto-fallback when MongoDB is unavailable
- UUID-based primary keys
- Full CRUD operations support

Environment Management

- **Python-dotenv 1.0.0** - Environment variable management

SSL/TLS Certificate Management

- **Certifi** - Trusted CA bundle for SSL verification

3. Machine Learning & Data Science

Core ML Libraries

- **Scikit-learn 1.4.2** - Complete ML framework
- **Algorithms Used:**
 - Random Forest Classifier (Primary model - 76.61% accuracy)
 - Logistic Regression
 - Decision Trees
 - Support Vector Machines
 - K-Nearest Neighbors
- **Preprocessing:**
 - StandardScaler - Feature normalization
 - LabelEncoder - Categorical encoding
- **Model Selection:**
 - train_test_split - Data splitting (80/20)
 - Cross-validation - Model validation
- **Evaluation Metrics:**
 - Accuracy Score
 - Precision, Recall, F1-Score
 - Confusion Matrix
 - ROC-AUC Score

Data Handling

- **Pandas 2.2.2** - Data manipulation and analysis
 - DataFrame operations
 - CSV file processing
 - Data cleaning and transformation
- **NumPy 1.26.4** - Numerical computing
 - Array operations
 - Mathematical functions
 - Statistical calculations

Class Imbalance Handling

- **Imbalanced-learn 0.12.3** - SMOTE (Synthetic Minority Over-sampling Technique)
 - Balances training data
 - Improves minority class prediction

Model Persistence

- **Joblib 1.3.2** - Efficient model serialization
- Saved Models:
 - `random_forest_model.pkl` - Trained RF classifier
 - `scaler.pkl` - Fitted StandardScaler
 - `label_encoder.pkl` - Categorical encoder
 - `feature_columns.pkl` - Feature metadata

4. Feature Engineering (20 Engineered Features)

Academic Performance Metrics

avg_approved - Average approved units across semesters
avg_grade - Average grade across semesters
total_approved - Total approved curricular units
approval_ratio_1st - First semester approval efficiency
approval_ratio_2nd - Second semester approval efficiency
grade_per_approved_1st - First semester grade efficiency
grade_per_approved_2nd - Second semester grade efficiency
improvement_rate - Semester-to-semester improvement

Enrollment & Course Load

total_enrolled - Total enrolled units
total_evaluated - Total evaluated units
completion_rate - Overall course completion rate

Academic Trajectory

grade_trend - Grade improvement/decline trend
enrollment_trend - Enrollment pattern changes
evaluation_trend - Evaluation completion patterns

Demographic & Financial

age_at_enrollment - Student age at enrollment
scholarship_binary - Scholarship holder status
debt_binary - Financial debt indicator
displaced_binary - Geographic displacement status

Combined Risk Indicators

financial_risk - Debt + No scholarship combination
academic_engagement - Approved units + Grades composite score

5. Development Tools

Code Quality & Linting

- **ESLint 9.9.1** - JavaScript/TypeScript linting
- **@eslint/js 9.9.1** - ESLint JavaScript config
- **eslint-plugin-react-hooks 5.1.0-rc.0** - React Hooks rules
- **eslint-plugin-react-refresh 0.4.11** - Fast Refresh support
- **TypeScript-ESLint 8.3.0** - TypeScript linting integration
- **Globals 15.9.0** - Global variable definitions

Build Tools

- **@vitejs/plugin-react 4.3.1** - Vite React plugin
- **Vite 5.4.2** - Fast HMR and optimized builds

Type Definitions

- **@types/react 18.3.5** - React type definitions
- **@types/react-dom 18.3.0** - ReactDOM types
- **@types/react-router-dom 5.3.3** - Router types

6. Deployment & Hosting

Frontend Hosting

- **Platform:** Netlify
- **URL:** <https://mentoraid.netlify.app>
- **Build Command:** `npm run build`
- **Publish Directory:** `dist`
- **Node Version:** 18
- **Features:**
 - Auto-deployment on GitHub push
 - SPA routing with redirects
 - HTTPS enabled
 - CDN distribution

Backend Hosting

- **Platform:** Render
- **URL:** <https://mentoraid-ai-based-student-drop-out.onrender.com>
- **Runtime:** Python 3.11.9
- **WSGI Server:** Gunicorn
- **Configuration:**
 - Workers: 4
 - Timeout: 120 seconds
 - Graceful timeout: 30 seconds
- **Features:**
 - Auto-deployment on GitHub push
 - Environment variables management
 - Persistent SQLite storage
 - Health check endpoints

7. API Architecture

Authentication Endpoints

- **POST** /api/auth/google - Google OAuth login
- **POST** /api/auth/refresh - Refresh access token
- **GET** /api/auth/me - Get current user info

Student Management

- **GET** /api/students - Fetch all students
- **GET** /api/students/:id - Get student details
- **POST** /api/students - Create new student
- **PUT** /api/students/:id - Update student
- **DELETE** /api/students/:id - Delete student

Prediction Endpoints

- **POST** /api/predict/single - Single student prediction
- **POST** /api/predict/batch - Batch CSV upload (up to 1000 students)

Intervention Tracking

- **GET** /api/interventions/:studentId - Get intervention history
- **POST** /api/interventions - Log new intervention

8. Security Features

Authentication Security

- JWT token-based authentication
- HTTP-only cookies for refresh tokens
- Google OAuth 2.0 integration
- Token expiration and refresh mechanism

API Security

- CORS configuration with whitelisted origins
- Request validation and sanitization
- Error handling without sensitive data exposure
- Rate limiting on prediction endpoints

Data Security

- Environment variables for sensitive credentials
- Encrypted database connections (MongoDB Atlas)
- SSL/TLS for all API communications
- Password-less authentication via Google OAuth

9. Performance Optimizations

Frontend Performance

- Code splitting with `React.lazy()`
- Vite's optimized production builds
- Tree-shaking for minimal bundle size
- CDN-based static asset delivery
- Responsive images and lazy loading

Backend Performance

- Unicorn multi-worker process model
- Database connection pooling
- Efficient model loading (load once at startup)
- Batch prediction optimization
- Caching of frequently accessed data

ML Model Optimization

- Pre-trained model loading (no runtime training)
- Numpy vectorization for fast predictions
- Efficient feature engineering pipeline
- Minimal preprocessing overhead

10. Mobile Responsiveness

Breakpoints

- **Mobile:** 320px - 639px (sm)
- **Tablet:** 640px - 767px (md)
- **Desktop:** 768px - 1023px (lg)
- **Large Desktop:** 1024px+ (xl)

Responsive Components

- Dashboard header (sticky, compact on mobile)
- Statistics cards (1-col mobile, 4-col desktop)
- Student table (horizontal scroll, progressive column disclosure)
- File upload controls (stacked on mobile, inline on desktop)
- Charts (responsive dimensions, touch-friendly)

11. Database Schema

Users Collection/Table

```
{
  id/ID: UUID (SQLite) or ObjectId (MongoDB)
  email: String (unique, indexed)
  name: String
  picture: String (profile photo URL)
  created_at: DateTime
  last_login: DateTime
  role: String (default: "teacher")
}
```

Students Collection/Table

```
{
  id/_id: UUID/ObjectId
  student_id: String (unique identifier)
  name: String
  email: String
  class: String
  department: String
  attendance: Float (0-100)
  marks: Float
  fee_status: String ("paid"/"pending")
  risk_level: String ("low"/"medium"/"high")
  risk_score: Float (0-100)
  created_by: UUID (foreign key to users)
  created_at: DateTime
  updated_at: DateTime
}
```

Predictions Collection/Table

```
{
  id/_id: UUID/ObjectId
  student_id: UUID (foreign key)
  user_id: UUID (foreign key to users)
  prediction: String ("Graduate"/"Dropout"/"Enrolled")
  confidence: Float (0-1)
  features: JSON (input features used)
  timestamp: DateTime
}
```

Interventions Collection/Table

```
{
  id/_id: UUID/ObjectId
  student_id: UUID (foreign key)
}
```

```
intervention_type: String
description: Text
date: DateTime
outcome: String
created_by: UUID (foreign key to users)
}
```


12. Model Performance Metrics

Best Model: Random Forest with SMOTE

- **Training Accuracy:** 98.84%
- **Test Accuracy:** 76.61%
- **Precision:** 0.77 (weighted avg)
- **Recall:** 0.77 (weighted avg)
- **F1-Score:** 0.76 (weighted avg)

Class-wise Performance

- **Graduate:** Precision 0.82, Recall 0.88
- **Dropout:** Precision 0.77, Recall 0.61
- **Enrolled:** Precision 0.67, Recall 0.74

Model Hyperparameters

```
RandomForestClassifier(  
    n_estimators=100,  
    max_depth=20,  
    min_samples_split=5,  
    min_samples_leaf=2,  
    random_state=42,  
    class_weight='balanced'  
)
```

13. Environment Variables

Frontend (.env)

```
VITE_API_URL=https://mentoraid-ai-based-student-drop-out.onrender.com  
VITE_GOOGLE_CLIENT_ID=284543669595-3bat01kl1t0g0uh41fgatmcb17hv3cla.apps.googleusercontent.com
```

Backend (.env)

```
MONGODB_URI=mongodb+srv://...  
MONGODB_DB_NAME=mentoraid  
JWT_SECRET_KEY=<secret-key>  
FLASK_ENV=production  
GOOGLE_CLIENT_ID=284543669595-3bat01kl1t0g0uh41fgatmcb17hv3cla.apps.googleusercontent.com  
CORS_ORIGINS=https://mentoraid.netlify.app
```

14. Project Structure

```
MentorAid/
├── backend/
│   ├── app.py # Flask application
│   ├── auth.py # Authentication helpers
│   ├── config.py # Configuration
│   ├── database.py # MongoDB models
│   ├── database_sqlite.py # SQLite fallback
│   ├── requirements.txt # Python dependencies
│   └── render.yaml # Render deployment config
├── ml-models/
│   ├── train_final_model.py # Model training script
│   ├── random_forest_model.pkl # Trained model
│   ├── scaler.pkl # Feature scaler
│   ├── label_encoder.pkl # Label encoder
│   └── feature_columns.pkl # Feature metadata
├── src/
│   ├── components/ # React components
│   │   ├── dashboard/ # Dashboard widgets
│   │   ├── student/ # Student detail views
│   │   ├── introduction/ # Landing page
│   │   └── ui/ # Reusable UI components
│   ├── contexts/ # React Context providers
│   ├── pages/ # Route pages
│   ├── types/ # TypeScript types
│   ├── App.tsx # Root component
│   └── main.tsx # Entry point
├── package.json # Node dependencies
├── tsconfig.json # TypeScript config
├── vite.config.ts # Vite config
├── tailwind.config.js # Tailwind CSS config
└── netlify.toml # Netlify deployment config
```

15. Key Features Implementation

1. Real-time Dropout Prediction

- **Technology:** Scikit-learn Random Forest
- **Input:** 35+ academic and demographic features
- **Output:** Dropout risk (Low/Medium/High) + Confidence score
- **Processing:** <500ms for single prediction, <5s for 1000 students

2. Batch CSV Upload

- **Technology:** Pandas DataFrame processing
- **Format:** CSV with 35 feature columns
- **Validation:** Schema validation, data type checking
- **Limit:** 1000 students per upload
- **Features:** Progress tracking, error reporting

3. Interactive Dashboards

- **Technology:** Recharts + React
- **Visualizations:**
 - Risk level distribution (pie chart)
 - Attendance trends (line chart)
 - Grade distribution (bar chart)
 - Student progress timeline (area chart)

4. Student Detail Pages

- **Technology:** React Router + Axios
- **Features:**
 - Academic metrics (attendance, grades, risk score)
 - Prediction history timeline
 - Intervention logs
 - AI-powered insights
 - Progress charts

5. Dark/Light Mode

- **Technology:** React Context + Tailwind CSS
- **Implementation:** System preference detection + manual toggle
- **Persistence:** LocalStorage

6. Google OAuth Login

- **Technology:** @react-oauth/google + Flask JWT

- **Flow:**

Frontend initiates Google OAuth

Backend verifies token with Google API

Create/update user in database

Issue JWT access + refresh tokens

Store tokens in httpOnly cookies

16. Testing & Quality Assurance

Testing Tools Used

- Manual testing across devices (mobile, tablet, desktop)
- Browser testing (Chrome, Firefox, Safari, Edge)
- API testing with Postman
- Database connectivity testing
- Model accuracy validation on test dataset

Quality Metrics

- ■ 76.61% ML model accuracy
- ■ 100% API endpoint uptime (with fallback)
- ■ Mobile responsive (320px - 4K screens)
- ■ <2s page load time
- ■ 0 critical security vulnerabilities

17. Future Enhancements (Planned)

Technology Additions

- **Redis** - Caching layer for predictions
- **PostgreSQL** - Production-grade relational database
- **Docker** - Containerization
- **Kubernetes** - Orchestration
- **TensorFlow** - Deep learning models
- **Apache Kafka** - Real-time event streaming
- **Grafana** - Monitoring dashboards

Feature Additions

- Email notifications for at-risk students
- SMS alerts via Twilio
- WhatsApp integration
- Advanced analytics with PowerBI
- Mobile app (React Native)
- Parent portal
- Multi-language support (i18n)

Summary

MentorAid is a production-ready, full-stack AI application combining modern web technologies with machine learning to solve a critical educational problem. The tech stack is carefully chosen for:

- **Performance:** Fast predictions, responsive UI
- **Scalability:** Cloud hosting, efficient database design
- **Reliability:** Database fallback, error handling
- **Security:** OAuth 2.0, JWT, HTTPS
- **Maintainability:** TypeScript, modular architecture
- **User Experience:** Mobile-first, dark mode, intuitive design

Total Technologies Used: 50+ libraries, frameworks, and tools across frontend, backend, ML, deployment, and development workflows.